

std::vector

Класс vector представляет собой **динамический массив**, который может увеличиваться по мере необходимости, чтобы хранить свои элементы.

Класс vector обеспечивает **произвольный доступ** к своим элементам через `operator[]`, а вставка и удаление элементов с конца вектора, как правило, происходит быстро

```
int main() {  
    std::vector<int> vect;  
    for (int count = 0; count < 6; ++count)  
        vect.push_back(10 - count); // вставить в конец  
  
    for (int count = 0; count < 6; ++count)  
        std::cout << vect[count] << ' '  
}  

```

Инициализация вектора

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> data = {1, 2, 3, 4, 5};
    for (int count = 0; count < 5; ++count) {
        std::cout << data[count] << ' ';
    }
}
```

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> i1; // пустой вектор целых
    std::vector<int> i2(5); // вектор из пяти целых
    std::vector<int> i3(5, 6); // вектор из пяти целых, заполненных "6"
    for (int count = 0; count < 5; ++count) {
        std::cout << i3[count] << ' ';
    }
}
```

Доступ к элементам

Доступ к элементам осуществляется оператором []:

```
std::vector<int> i3(5, 6); // вектор из пяти целых, заполненных "6"  
i3[3] = 4;
```

Также, как и у array, в векторе можно использовать функцию at()

```
std::cout << i3.at(7);
```

Также, как и у array, в векторе можно использовать функции front() и back()

```
std::cout << i3.front() << '\n';  
std::cout << i3.back() << '\n';
```

Добавление и удаление элементов в векторе

В вектор можно добавлять элементы в конец и удалять их с конца. Для этого существуют функции `push_back` и `pop_back`:

```
#include <iostream>
#include <vector>

int main() {
    int x;
    std::vector<int> data;
    while (std::cin >> x && x >= 0) {
        data.push_back(x); // добавляем очередное число в вектор
    }
    while (!data.empty()) { // пока вектор не пуст
        std::cout << data.back(); // Выводим последний элемент
        data.pop_back(); // и удаляем его
    }
}
```

Резерв памяти

Вектор хранит элементы в памяти в виде непрерывной последовательности. При этом в конце последовательности резервируется дополнительное место для быстрого добавления новых элементов. Когда этот резерв заканчивается, при вставке очередного элемента происходит *реаллокация*: элементы вектора копируются в новый, более просторный блок памяти.

Доказано, что если размер нового блока выбирать в два раза больше предыдущего размера, то амортизационная сложность добавления элемента будет константной.

Текущий резерв вектора можно узнать с помощью функции `capacity`.

resize or reserve

- **resize** используется, если нужно немедленно инициализировать новые элементы;
- **reserve** — если нужно избежать затрат на инициализацию и только выделить память для будущего роста.

```
template <typename T>
class vector {
    T* arr;
    size_t sz;
    size_t cap;
public:
    void reserve(size_t newcap) {
        T* newarr = new T[newcap];    //А что, если нет конструктора по умолчанию?
        for (size_t i; i < sz; ++i) {
            newarr[i] = arr[i];        //Или оператора присваивания?
        }
        delete [] arr;
        arr = newarr;
        cap = newcap;
    }
    void push_back(const T& value) {
        if (sz == cap) {
            reserve(cap > 0 ? cap * 2 : 1);
        }
        //construct
    }
};
```

placement new

Размещение на ранее выделенной памяти используется для оптимизации, когда нужно создать много объектов. Вместо постоянного перераспределения памяти каждый раз, когда нужен новый экземпляр, более эффективным будет выполнить единое выделение фрагмента памяти, который может хранить несколько объектов, даже если в данный момент не планируется использовать их всех.

```
// Statically allocate the storage with automatic storage duration
// which is large enough for any object of type "T".
char buf[sizeof(T)];
T* tptr = new(buf) T; // Construct a "T" object, placing it directly into your
                      // pre-allocated storage at memory address "buf".
tptr->~T();           // You must manually call the object's destructor
                      // if its side effects is depended by the program.
```

```
char *buf = new char[sizeof(string)]; // pre-allocated buffer
std::string *p = new (buf) string("hi"); // placement new
```

Резервирование памяти сводится к

1. Выделению «сырой» памяти и, если нужно,
2. Инициализации объектов на этой памяти

```
void *raw = malloc(sizeof(T)); // любой способ выделения памяти  
T *pt = new(raw) T(параметры); // инициализация объекта
```

Освобождение памяти

Удаление следует выполнять в обратном порядке, т. е. необходимо вызвать деструктор и функцию освобождения "сырой" памяти:

```
pt->~T();  
free(pt);
```


Причины использования placement new

- Контроль над выделением памяти
- Скорость создания объектов. Они просто конструируются на заранее выделенной памяти
- Повторное использования памяти. На том же выделенном фрагменте можно сконструировать другие объекты

Поэтому вместо прямого создания объектов, лучше просто выделить под них «сырую» память:

```
T* newarr = new T[newcap]; //???
```



```
T* new_arr = static_cast<T*>(operator new (new_cap*sizeof(T)));
```

А на выделенной памяти уже конструировать объекты:

```
for (size_t i; i < sz; ++i) {  
    newarr[i] = arr[i]; } //???
```



```
for (size_t i = 0; i < sz; ++i) {  
    new(new_arr + i) T(arr[i]);}
```

Но и освобождение памяти тогда надо делать по-другому:

```
for (size_t i = 0; i < sz; ++i) {  
    (arr + i)->~T();  
}  
operator delete(arr);
```

Аллокаторы

Это специальные классы, которые используются для управления памятью в контейнерах. Простой аллокатор:

```
template<typename T>
class SimpleAllocator {
public:
    T* allocate(std::size_t n) {
        if (auto p = static_cast<T*>(std::malloc(n * sizeof(T)))) {
            return p;
        }
        throw std::bad_alloc();
    }

    void deallocate(T* p, std::size_t) noexcept {
        std::free(p);
    }

    template<typename U, typename... Args>
    void construct(U* p, const Args&&... args) {
        new(p) U(args...);
    }

    template<typename U>
    void destroy(U* p) noexcept {
        p->~U();
    }
};
```

Существует много видов аллокаторов

- Liner allocator
- Pool allocator
- Stack allocator
- FreeList allocator
- и др

Все контейнеры принимают вторым шаблонным аргументом аллокатор, причём, самый простой

```
template <typename T, typename Allocator = std::allocator<T>>
class vector {
    // . . .
};
```

Но обращаются к аллокатору через allocator_traits

```
Allocator alloc;
using Alloc_traits = std::allocator_traits<Allocator>;
// . . .
Alloc_traits::construct(alloc, new_arr + i, arr[i]);
```

Шаблонный класс allocator_traits обеспечивает стандартизированный доступ к различным методам аллокаторов. Принимая в качестве шаблонного параметра аллокатор, этот класс «проверяет» наличие определённых методов. Поэтому все стандартные контейнеры обращаются к аллокаторам только через allocator_traits.

Использование принципа "выделение памяти отдельно от конструирования объектов" (который реализуется через аллокатор) позволяет в операторе равенства избежать излишнего перевыделения памяти

```
if (capacity >= other.size) {  
    // используется аллокатор только для construct/destroy  
    for (size_t i = 0; i < size; ++i)  
        allocator_traits::destroy(alloc, buffer + i);  
  
    for (size_t i = 0; i < other.size; ++i)  
        allocator_traits::construct(alloc, buffer + i, other[i]);  
  
    size = other.size;  
}
```

?

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v(5,6);
    int* p = &v[3];
    *p = 2;
    std::cout << *p << '\n';

    v.push_back(1);
    std::cout << *p << '\n';
}
```

Cppcheck бдит 😊

```
int main() {  
    std::vector<int> v(5,6);  
    int* p = &v[3];  
    *p = 2;  
    std::cout << *p  
    v.push_back(1);  
    std::cout << *p << '\n';  
}
```

Using pointer to local variable 'v' that may be invalid. Cppcheck(invalidContainer)

[View Problem \(F8\)](#) No quick fixes available

Это явление называется инвалидация указателей (с ссылками будет то же самое)

std::deque

double-ended queue (двусторонняя очередь).

Если вектор располагает элементы в памяти непрерывно, то std::deque располагает их кусочно-непрерывно, в отдельных *страницах* (непрерывных блоках) памяти фиксированного размера. Но даже для хранения одного элемента в деке будет выделена целая страница. Сами страницы не обязательно расположены в памяти подряд. Отдельно поддерживается перечень указателей на начала страниц. Размеры страниц зависят от sizeof(T) и от конкретной реализации дека.

std::deque

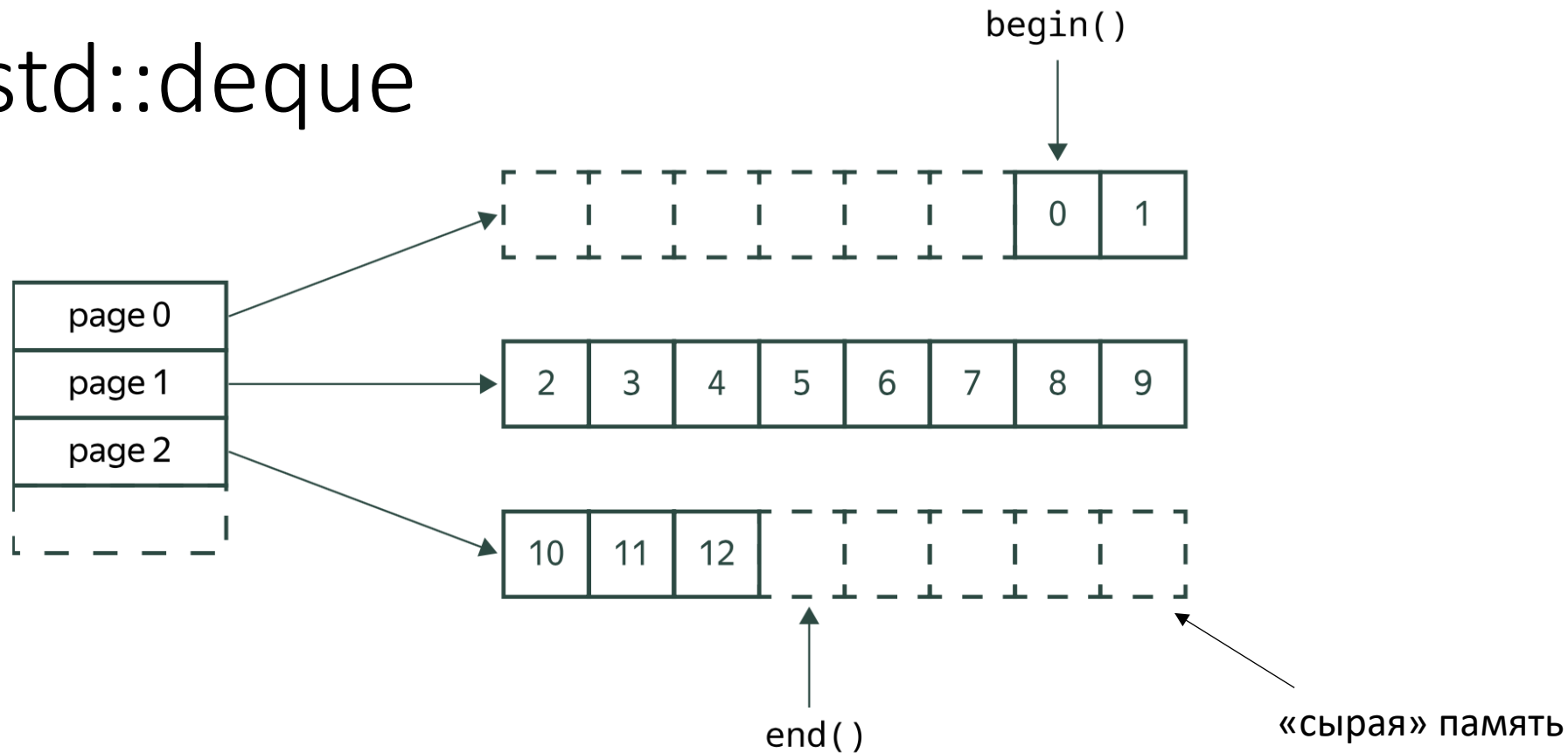
```
#include <deque>
#include <iostream>

int main() {
    std::deque<int> deq;
    for (int count = 0; count < 3; ++count) {
        deq.push_back(count); // вставить в конец
        deq.push_front(10 - count); // вставить в начало
    }

    for (int index = 0; index < deq.size(); ++index)
        std::cout << deq[index] << ' ';

}
```

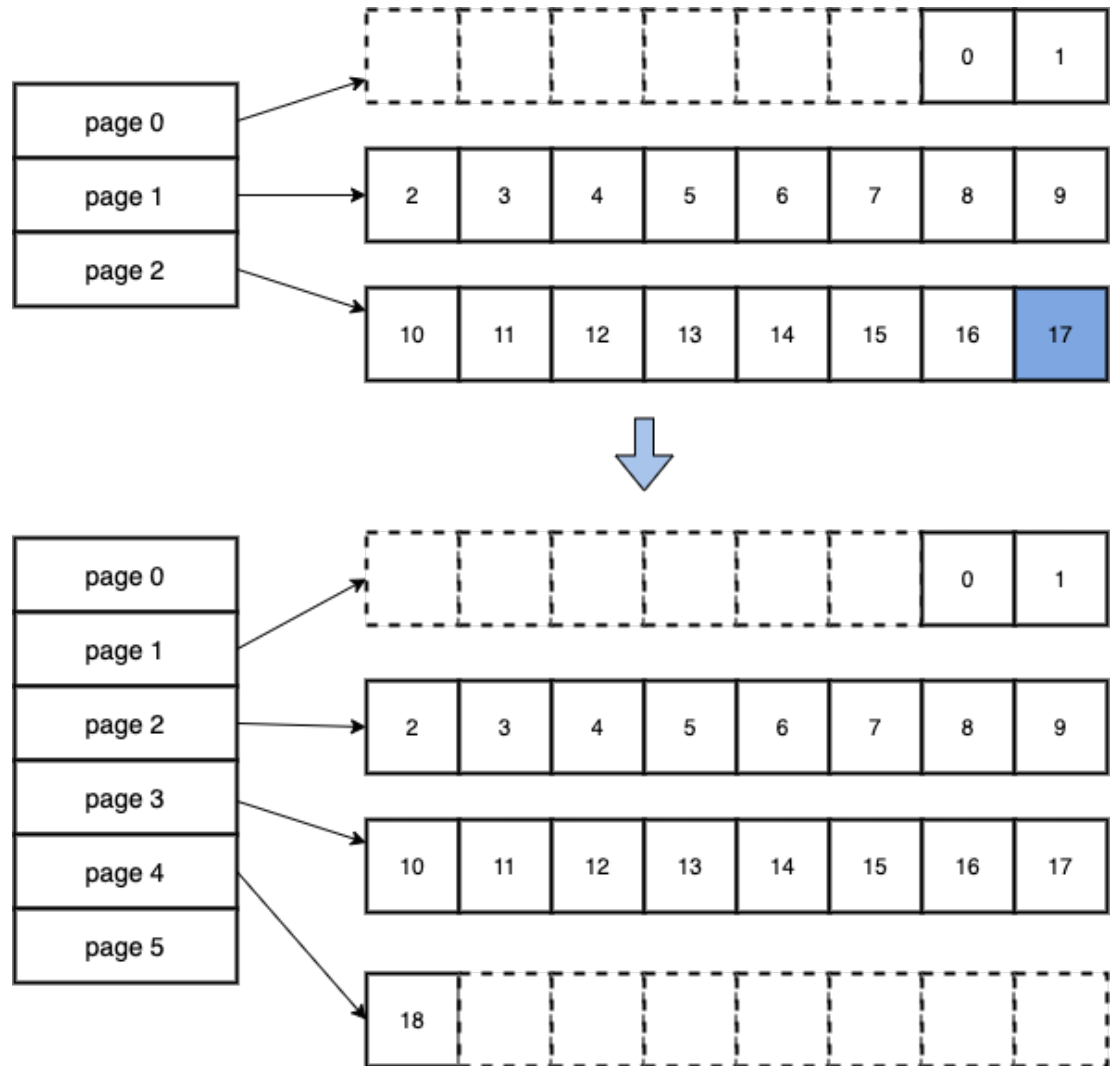
std::deque



В отличие от вектора, дек гарантирует, что при вставке или удалении по краям элементы останутся в тех же самых ячейках памяти, что и были. Вставка в середину дека и удаление из неё уже требуют сдвига элементов.

Дек поддерживает обращение к элементу по индексу за $O(1)$. Для обращения к элементу деку приходится делать два разыменования указателей.

Расширение deque



Таким образом, при работе с первым и последним элементом в деке не будет инвалидации указателей (в отличии от вектора)

std::list

- Это особый тип контейнера последовательности, называемый двусвязным списком, где каждый элемент в контейнере содержит указатели, указывающие на следующий и предыдущий элементы в списке. Списки обеспечивают доступ только к началу и концу списка – **произвольного доступа нет**.
- Преимущество списков в том, что вставка элементов в список происходит очень быстро, если уже известно, куда их надо вставить. Для просмотра списка обычно **используются итераторы (попозже)**.

std::list

```
#include <iostream>
#include <list>

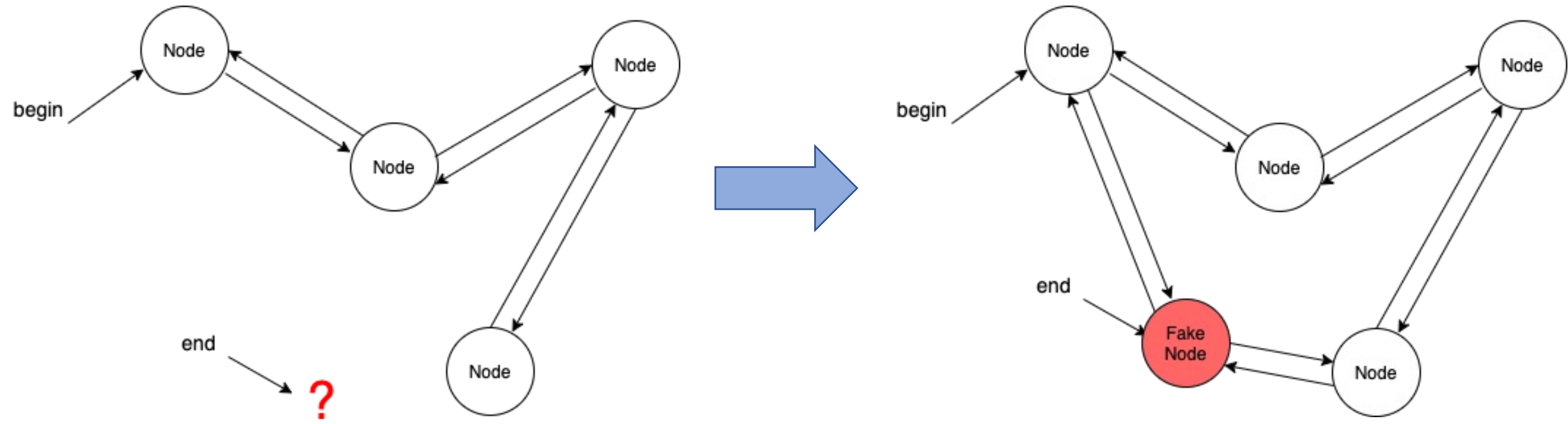
int main() {
    // Создание списка целых чисел
    std::list<int> numbers = {1, 2, 3, 4, 5};

    // Добавление элемента в конец списка
    numbers.push_back(6);
    numbers.push_back(5);

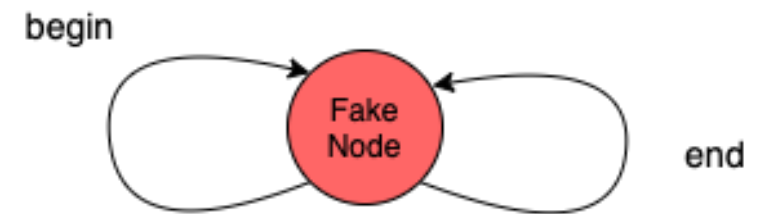
    // Вывод последнего элемента
    std::cout << *(--(numbers.end()));

    // Вывод элементов списка
    std::cout << "List elements: ";
    for (const auto& num : numbers) {
        std::cout << num << " ";
    }
}
```

Как организовать перемещение по списку?



Если список пуст, в нём должен быть один пустой узел:



Как-то так

```
template <typename T, typename Allocator = std::allocator<T>>
class list {
    struct base_node {
        base_node* prev;
        base_node* next;
    };
    struct node: base_node {
        T value;
    };
    base_node fake_node;
    size_t sz;
public:
    list(): fake_node {&fake_node, &fake_node}, sz(0){}
    // . . .
};
```


Modifiers

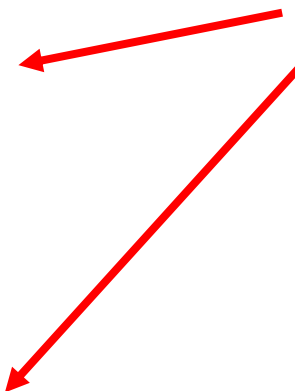
clear	clears the contents (public member function)
insert	inserts elements (public member function)
insert_range (C++23)	inserts a range of elements (public member function)
emplace (C++11)	constructs element in-place (public member function)
erase	erases elements (public member function)
push_back	adds an element to the end (public member function)
emplace_back (C++11)	constructs an element in-place at the end (public member function)
append_range (C++23)	adds a range of elements to the end (public member function)
pop_back	removes the last element (public member function)
push_front	inserts an element to the beginning (public member function)
emplace_front (C++11)	constructs an element in-place at the beginning (public member function)
prepend_range (C++23)	adds a range of elements to the beginning (public member function)
pop_front	removes the first element (public member function)
resize	changes the number of elements stored (public member function)
swap	swaps the contents (public member function)

Operations

merge	merges two sorted lists (public member function)
splice	moves elements from another list (public member function)
remove remove_if	removes elements satisfying specific criteria (public member function)
reverse	reverses the order of the elements (public member function)
unique	removes consecutive duplicate elements (public member function)
sort	sorts the elements (public member function)

Основные методы list

У `std::list` реализована своя сортировка, стандартный алгоритм сортировки не подходит для списка (по причине использования другого типа итератора)



Итераторы

В некоторых контейнерах нет возможности получить доступ к элементу по индексу, то есть нельзя использовать целочисленные индексы.

Итератор – это такой тип, который позволяет делать обход последовательности. Его можно разыменовывать, инкрементировать, а также сравнивать друг с другом на равенство. Можно сказать, что простой указатель – это тоже итератор.

Любой контейнер можно обходить итератором.

Итератор лучше всего представить себе как **указатель на заданный элемент** в контейнере с набором перегруженных операторов для предоставления набора четко определенных функций:

- **operator*** – **разыменование итератора** возвращает элемент, на который итератор в данный момент указывает;
- **operator++** – перемещает итератор **к следующему элементу** в контейнере. Большинство итераторов также предоставляют **operator--** для перехода **к предыдущему элементу**;
- **operator==** и **operator!=** – базовые **операторы сравнения**, чтобы определить, указывают ли два итератора на один и тот же элемент.

- **operator=** – присваивание итератору новой позиции (обычно в начало или в конец элементов контейнера).

Каждый контейнер включает четыре основные функции-члена для использования с **operator=**:

- **begin()** возвращает итератор, представляющий начало элементов в контейнере;
- **end()** возвращает итератор, представляющий элемент сразу за концом элементов (**не указывает на последний элемент в контейнере!**), итерация по элементам может продолжаться до тех пор, пока итератор не достигнет **end()**.

- **cbegin()** возвращает константный (только для чтения) итератор, представляющий начало элементов в контейнере;
- **cend()** возвращает константный (только для чтения) итератор, представляющий элемент сразу за концом элементов.

Все контейнеры предоставляют (как минимум) два типа итераторов:

- **container::iterator** предоставляет итератор для чтения/записи;
- **container::const_iterator** предоставляет итератор только для чтения

Категории итераторов

Для каждого контейнера конкретный тип итератора будет отличаться. Так, итератор для контейнера `list<int>` представляет тип `list<int>::iterator`, а итератор контейнера `vector<int>` представляет тип `vector<int>::iterator` и так далее. Однако общий функционал, который применяется для доступа к элементам, будет аналогичен

- Input Iterator. Самый простой итератор. Его можно разыменовывать, инкрементировать, а также сравнивать на равенство.
- Forward Iterator. Умеет всё то же, что и Input, но также может быть скопирован.
- Bidirectional Iterator. Умеет делать всё то же, что и Forward, но также перемещаться «назад» (`list`, `map`).
- RandomAccess Iterator. Умеет всё, что и простые указатели (`+=n`, `-=n` и т.д.) – `array`, `vector`, `deque`
- Output Iterator. Итератор, в который можно записывать значения.

```

template <typename T, typename Allocator = std::allocator<T>>
class vector {
    T* arr;
    size_t sz;
    size_t cap;
    Allocator alloc;
    using Alloc_traits = std::allocator_traits<Allocator>;
public:
    class iterator {
    private:
        T* ptr;
    public:
        iterator (T* ptr): ptr(ptr) {}
        iterator (const iterator&) = default;
        iterator& operator =(const iterator&) = default;
        T& operator* () { return *ptr;}
        T* operator ->() {return ptr;}
        iterator& operator++() {
            ++ptr;
            return *this;
        }
        iterator operator++(int) {
            iterator tmp = *this;
            ++ptr;
            return tmp;
        }
    };
    iterator begin() { return iterator(arr);}
    iterator end() {return iterator(arr + sz);}
    // . . .
};

```

```

int main (){
    vector<int> v(2);
    v.push_back(4);
    vector<int>::iterator x1 = v.begin();
    std::cout << *x1;
}

```

Упрощённый пример
vector<int>::iterator

Цикл range-based for

Range-based for (или *«цикл, основанный на диапазоне»*) предоставляет более простой и безопасный способ итерации по массиву (или по любому контейнеру).

Синтаксис цикла следующий:

```
for (объявление_элемента : контейнер)  
    стейтмент;
```

Выполняется итерация по каждому элементу контейнера, присваивая значение текущего элемента переменной, объявленной как **элемент** (объявление_элемента). В целях улучшения производительности объявляемый элемент должен быть того же типа, что и элементы контейнера, иначе произойдет неявное преобразование.

for (auto number: array)

Поскольку объявляемый элемент цикла range-based for должен быть того же типа, что и элементы контейнера, то лучше использовать ключевое слово `auto`, тогда компилятор сам вычисляет тип данных элементов.

```
int main() {  
    int array[] = { -1, 7, 4, 5, 30, 13 };  
    for (const auto& number : array)  
        std::cout << number << ' '  
}
```

Каждый обработанный элемент копируется в переменную `number`. Чтобы избежать затрат на копирование, лучше пользоваться константной ссылкой.

Цикл range-based for – это синтаксический сахар 😊

По-настоящему это выглядит приблизительно так (а до C++11 можно было только так):

```
std::vector<int> v;  
v.push_back(4);  
v.push_back(6);  
v.push_back(3);  
  
for(std::vector<int>::iterator it = v.begin(); it != v.end(); ++it) {  
    std::cout << *it << '\n';  
}
```

https://cppinsights.io

```
for(std::vector<int>::iterator it = v.begin(); it != v.end(); ++it) {  
    std::cout << *it << '\n';  
}
```



[C++ Standard: C++ 17]



Defa... ▼

More

[New C++ Insights Episode](#) ×

Made by [Andreas Fertig](#)
Powered by [Flask](#) and [CodeMirror](#)

Source:

Insight:

```
1 #include <cstdio>  
2 #include <iostream>  
3 #include <vector>  
4  
5 int main() {  
6     std::vector<int> v;  
7     v.push_back(4);  
8     v.push_back(6);  
9     v.push_back(3);  
10    for(std::vector<int>::iterator it = v.begin(); it != v.end(); ++it) {  
11        std::cout << *it << '\n';  
12    }  
13 }
```

```
1 #include <cstdio>  
2 #include <iostream>  
3 #include <vector>  
4  
5 int main()  
6 {  
7     std::vector<int, std::allocator<int> > v = std::vector<int, std::allocator<int> >();  
8     v.push_back(4);  
9     v.push_back(6);  
10    v.push_back(3);  
11    for(__gnu_cxx::__normal_iterator<int *, std::vector<int, std::allocator<int> > > it = v.begin(); __gnu_cxx::__operator!=(it, v.end()); it.operator++()) {  
12        std::operator<<(std::cout.operator<<(it.operator*()), '\n');  
13    }  
14  
15    return 0;  
16 }  
17
```

https://cppinsights.io

```
std::vector<int> v;
v.push_back(4);
v.push_back(6);
v.push_back(3);
for(int x: v) {
    std::cout << x << '\n';
}
```



[C++ Standard: C++ 17]



Defa... ▾

More

[New C++ Insights Episode](#) ×

Made by [Andreas Fertig](#)
Powered by [Flask](#) and [Code](#)

Source:

```
1 #include <cstdio>
2 #include <iostream>
3 #include <vector>
4
5 int main() {
6     std::vector<int> v;
7     v.push_back(4);
8     v.push_back(6);
9     v.push_back(3);
10    for(int x: v) {
11        std::cout << x << '\n';
12    }
13 }
```

Insight:

```
1 #include <cstdio>
2 #include <iostream>
3 #include <vector>
4
5 int main()
6 {
7     std::vector<int, std::allocator<int> > v = std::vector<int, std::allocator<int> >();
8     v.push_back(4);
9     v.push_back(6);
10    v.push_back(3);
11    {
12        std::vector<int, std::allocator<int> > & __range1 = v;
13        __gnu_cxx::__normal_iterator<int *, std::vector<int, std::allocator<int> > > __begin1 = __range1.begin();
14        __gnu_cxx::__normal_iterator<int *, std::vector<int, std::allocator<int> > > __end1 = __range1.end();
15        for(; __gnu_cxx::operator!=(__begin1, __end1); __begin1.operator++()) {
16            int x = __begin1.operator*();
17            std::operator<<(std::cout.operator<<(x), '\n');
18        }
19    }
20 }
21 return 0;
22 }
23
```

Так лучше не делать, потому что при push_back инвалидируются итераторы

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {3, 4, 5, 6, 7};
    for(int& x : v) {
        v.push_back(x);
    }

    for(int& x : v) {
        std::cout << x << ' ';
    }
}
```

Инвалидация итераторов

Некоторые операции над контейнерами делают существующие итераторы некорректными.

- В векторе инвалидируются все итераторы (и указатели), если изменяется `capacity`, то есть происходит реаллокация. Если реаллокации нет, все итераторы и ссылки валидны.
- В `deque` **удаление/добавление** инвалидирует все итераторы, в случае удаления/добавления первого или последнего элементов указатели не инвалидируются.

Инвализация итераторов и ссылок

Iterator invalidation

Read-only methods never [invalidate](#) iterators or references. Methods which modify the contents of a container may invalidate iterators and/or references, as summarized in this table.

Category	Container	After insertion , are...		After erasure , are...		Conditionally
		iterators valid?	references valid?	iterators valid?	references valid?	
Sequence containers	array	N/A		N/A		
	vector	No		N/A		Insertion changed capacity
		Yes		Yes		Before modified element(s) (for insertion only if capacity didn't change)
		No		No		At or after modified element(s)
	deque	No	Yes	Yes, except erased element(s)		Modified first or last element
			No	No		Modified middle only
	list	Yes		Yes, except erased element(s)		
	forward_list	Yes		Yes, except erased element(s)		
Associative containers	set multiset map multimap	Yes		Yes, except erased element(s)		
Unordered associative containers	unordered_set unordered_multiset unordered_map unordered_multimap	No	Yes	N/A		Insertion caused rehash
		Yes		Yes, except erased element(s)		No rehash

Так вот, std::list

```
#include <iostream>
#include <list>

int main() {
    // Создание списка целых чисел
    std::list<int> numbers = {1, 2, 3, 4, 5};

    // Добавление элемента в конец списка
    numbers.push_back(6);

    // Вставка элемента перед вторым элементом
    auto it = std::next(numbers.begin(), 1);
    numbers.insert(it, 10);

    // Удаление третьего элемента
    it = std::next(numbers.begin(), 2);
    numbers.erase(it);
    // Вывод элементов списка
    std::cout << "List elements: ";
    for (const auto& num : numbers) {
        std::cout << num << " ";
    }
}
```


std::string официально не принадлежит STL

Контейнер `std::string` можно рассматривать как особый случай вектора символов `std::vector<char>`. У строки есть все те же функции, что и у вектора (например, `pop_back` или `resize`). Но также есть специальные методы:

```
s.substr(pos)
```

Выделение подстроки, начиная с позиции `pos`, возвращает строку

```
s.substr(pos, count)
```

Выделение подстроки, начиная с позиции `pos` в количестве `count`, возвращает строку

```
s.find(' ', '');
```

Поиск символа, возвращает позицию результата поиска

```
s.find(' ', ' ', pos)
```

Возвращает позицию результата поиска, поиск осуществляется, начиная с `pos`

```
s.find("wor")
```

Поиск строки, возвращает начальную позицию найденной строки

```
#include <iostream>
#include <string>

int main() {
    std::string s = "Hello";

    // в строках переопределены операторы + и +=
    s += ','; //можно добавить символ в конец строки, это аналог push_back
    s += "world!"; // можно добавить всю строку
    std::cout << s << "\n"; // Hello,world!

    // подстрока "world" из 5 символов, начиная с 6-й позиции
    std::string sub1 = s.substr(6, 5);
    // подстрока "world!" с 6-й позиции и до конца
    std::string sub2 = s.substr(6);

    // поиск символа или подстроки
    size_t pos1 = s.find(','); // позиция первой запятой, в данном случае 5
    size_t pos2 = s.find(',', pos1 + 1); // попытка найти следующую запятую
    size_t pos3 = s.find("wor"); // вернётся 6
    if (s.find("#") == std::string::npos) {
        std::cout << "# not found";
    }
}
```

`std::string::npos` — это статическая константа класса `std::string`, представляющая максимально возможное значение для типа `size_t`. Эта константа часто используется для индикации отсутствия позиции символа или подстроки в строке.

В std::string есть insert, replace, erase

```
int main() {  
    std::string s = "Helloworld!";  
  
    // вставка подстроки  
    s.insert(5, ", ");  
    std::cout << s << "\n"; // Hello, world!  
  
    // замена указанного диапазона на новую подстроку  
    s.replace(0, 5, "Good bye");  
    std::cout << s << "\n"; // Good bye, world!  
  
    // удаление подстроки  
    s.erase(8, 7);  
    std::cout << s << "\n"; // Good bye!  
}
```

std::initializer_list (c C++11)

```
class string {
    char* arr = nullptr;
    size_t sz = 0;
    size_t cap = 0;
public:
    string(std::initializer_list<char> list)
        : arr(new char[list.size()])
        , sz(list.size())
        , cap(sz + 1) {
        std::copy(list.begin(), list.end(), arr);
    }
    // . . .
};

int main() {
    string s1 = {'g', 'e', 't'};
}
```

Адаптеры

Адаптеры над стандартными контейнерами — это не самостоятельные контейнеры. Они используют какой-нибудь другой контейнер (например, дек или вектор) для хранения своих элементов, но при этом предоставляют свой набор функций для работы с ними. В стандартной библиотеке есть адаптеры `std::stack`, `std::queue` и `std::priority_queue`.

std::stack

- Стек — это последовательность элементов, в которую разрешается добавлять и извлекать элементы только с одной стороны. Обращение к промежуточным элементам не допускается. Говорят, что структура данных «стек» реализует идею LIFO (last in — first out).
- Эти действия можно сделать с помощью вектора или дека (по умолчанию используется дек) и операций `push_back` и `pop_back`. Адаптер `std::stack` является обёрткой над такими контейнерами с особым интерфейсом — функциями `push`, `pop` и `top`.
- Функция `pop()` ничего не возвращает. Функция `top()` возвращает элемент, но никак не меняет состояние стека. Так было сделано для гарантии безопасности (см. идиому RAI)

std::queue

Очередь реализует идею FIFO (first in — first out). Можно считать, что элементы встают в очередь с одного конца, а извлекаются с другого. Очередь предоставляет функции `push`, `pop`, `front` и `back`.

Так же как и в `std::stack`, в очереди по умолчанию используется `std::deque` для хранения элементов. Этот контейнер точно так же можно заменить. Для этого нужно указать тип нового контейнера во втором шаблонном параметре. Однако в отличие от стека здесь не получится использовать `std::vector`, так как у него нет функций `push_front` и `pop_front`.

std::priority_queue

Эта структура данных позволяет за логарифмическое время добавлять и удалять элементы и за константное время получать максимальный элемент. Типичная реализация очереди с приоритетами основана на структуре данных типа «куча». Для хранения элементов используется контейнер с индексами и с возможностью добавления в конец. Здесь можно использовать `std::vector` и `std::deque`. По умолчанию используется `std::vector`.

Адаптеры над итераторами – это обёртки над итераторами, которые изменяют поведение итераторов. Они упрощают работу с контейнерами и бывают очень полезны в стандартных алгоритмах. Адаптеры находятся в заголовочном файле `<iterator>`

std::reverse_iterator

Это итератор, перебирающий элементы в обратном направлении. Обратный итератор базируется на двунаправленном итераторе (BidirectionalIterator). После инкрементирования он будет указывать на предыдущий элемент контейнера.

```
#include <iostream>
#include <list>

int main() {
    std::list<int> list { 1, 2, 3, 4, 5 };
    std::list<int>::iterator it = list.end();
    std::list<int>::reverse_iterator rit = std::make_reverse_iterator(it);
    while (rit != list.rend()) {
        std::cout << *rit << ' ';
        ++rit;
    }
}
```

std::move_iterator

Данный итератор поддерживает семантику перемещения элементов контейнера.

```
#include <iostream>
#include <set>
#include <vector>

int main() {
    std::set<std::string> words { "mango", "orange", "apple", "pear" };
    std::vector<std::string> newWords { "cucumber", "tomato", "pumpkin" };
    words.insert(std::make_move_iterator(newWords.begin()),
                std::make_move_iterator(newWords.end()));

    std::cout << " words: "; //Будут выведены все слова
    for (auto word : words) {
        std::cout << '\'' << word << " ";
    }
    std::cout << "\nnewWords: "; //Будет пусто
    for (auto word : newWords) {
        std::cout << '\'' << word << " ";
    }
}
```

Потоковые итераторы

Потоковые итераторы позволяют работать с потоком как с контейнером и использовать стандартные алгоритмы.

Итератор потока ввода `'std::istream_iterator'` предназначен для чтения данных из объекта `'std::basic_istream'` (например, из `'std::cin'` или файла, открытого на чтение). При создании итератора потока ввода требуется инстанцировать его типом считываемых элементов и передать в конструктор поток.

Считывание элементов происходит в момент инкрементирования итератора через оператор '++'. Операция инкрементирования использует операцию '>>' потока для чтения. Операция разыменования '*' возвращает только константную копию элемента

```
#include <iostream>
#include <iterator>

int main() {
    std::istream_iterator<int> input_iter(std::cin);
    std::ostream_iterator<int> output_iter(std::cout, " ");
    //В данном случае ввод будет продолжаться до тех пор, пока на вход поступают int
    while (input_iter != std::istream_iterator<int>()) {
        if (*input_iter % 2 == 0) {
            *output_iter = *input_iter;
        }
        input_iter++;
    }
}
```

Итератором, который обозначает конец последовательности, является итератор по умолчанию 'std::istream_iterator<int>()'. Это вызов конструктора по умолчанию std::istream_iterator<int>, который создаёт специальный итератор. Он не привязан ни к какому потоку и служит маркером конца для сравнения с рабочими итераторами. Разыменовывание приводит к неопределённому поведению.

Output iterator (итераторы вставки)

Итераторы вставки заменяют операцию присваивания на операцию вставки. Это может быть полезно при добавлении группы новых элементов в контейнер. Существует три вида итераторов вставки:

'std::front_insert_iterator<Container>' – использует метод 'push_front' для вставки элементов в начало контейнера;

'std::back_insert_iterator<Container>' – использует метод 'push_back' для вставки элементов в конец контейнера;

'std::insert_iterator<Container, Iter>' – вставляет элемент в произвольную позицию контейнера, заданную итератором Iter. Output итераторы принимают контейнер.

```
#include <iostream>
#include <vector>

int main() {
    int a[6] = {3,7,2,9,5,1};
    std::vector<int> vec;
    std::copy (a, a + 6, std::back_insert_iterator<std::vector<int>>(vec));
    for (const auto&x : vec) {
        std::cout << x << ' ';
    }
}
```

Пример

```
#include <array>
#include <iostream>
#include <iterator>
#include <string>

int main() {
    std::array<int, 3> a1{1, 2, 3};
    std::array<int, 3> a2 = {1, 2, 3};
    std::array<std::string, 2> a3 = {std::string("a"), "b"};

    std::sort(a1.begin(), a1.end());
    std::reverse_copy(a2.begin(), a2.end(),
                     std::ostream_iterator<int>(std::cout, " ")); // В консоли: 3 2 1
    std::cout << '\n';

    for (const auto& s : a3)
        std::cout << s << ' '; // В консоли: a b
}
```

Ассоциативные контейнеры

- **Ассоциативные контейнеры** — это контейнеры, которые автоматически сортируют свои входные данные, когда эти входные данные вставляются в контейнер. По умолчанию ассоциативные контейнеры сравнивают элементы с помощью `operator<`.
- Ассоциативные контейнеры автоматически сортируют свои элементы **по некоторому критерию**. **Критерий** представляется в **виде функции**, которая сравнивает либо значения, либо специальные ключи, определяемые для этих значений.

std::map

Это упорядоченный ассоциативный массив ключ-значение. Для сортировки применяется КЧД. Среднее время работы вставки, удаления и поиска работает за $O(\log(n))$

```
#include <iostream>
#include <map>

int main() {
    std::map<char, int> f;
    f['a'] = 10;
    f['b'] = 30;

    std::map<char, int> s(f.begin(), f.end());
    std::map<char, int> t(s);

    for(const auto& element: f) {
        std::cout << element.first << ' ' << element.second << '\n';
    }

    std::cout << t['a'] << '\n';

    for(const auto& [el1, el2]: t) { //since C++17
        std::cout << el2 << '\n';
    }
}
```

Элементы `std::map`

Каждый элемент `map` фактически представляет объект типа **`std::pair<const Key, Value>`**, который хранит, как ключ, так и значение. С помощью полей `first` и `second` объекта можно получить, соответственно, ключ и значение элемента.

```
for(const auto& element: f) {  
    std::cout << element.first << ' ' << element.second << '\n';  
}
```

Итератор `std::map` представляет собой указатель на **`pair<const Key, Value>`**

Основные методы std::map

```
int main() {
    std::map<char, int> f { {'c', 20}, {'d', 80}, {'e', 60}};
    f['a'] = 10;
    f['b'] = 30;

    f.insert(std::pair('x', 52));
    f.insert(std::map<char, int>::value_type('c', 12));
    f.insert({'z', 34});

    //insert возвращает pair <iterator, bool>
    std::cout << f.insert(std::map<char, int>::value_type('a', 24)).second;

    //at() возвращает значение value
    std::cout << f.at('b');

    auto it = f.begin();
    //erase и find возвращают итераторы типа pair<const Key, Value>*
    std::cout << f.erase(++it)->first; //erase возвращает следующий за удалённым
    std::cout << f.find('a')->second;
}
```

operator[]

operator[] с несуществующим ключом всегда выполняет вставку элемента в контейнер значением по умолчанию:

```
std::map<int,int> m;  
std::cout << m[0] << ' ' << m[1] << ' ' << m[3]; // 0 0 0  
m[1] = 20; //при наличии такого ключа значение перезаписывается
```

operator[] не является константным:

```
template <typename T>  
void f(const T& t) {  
    std::cout << t[0];  
}  
  
int main() {  
    std::map<int,int> m;  
    m[0] = 0;  
    m[1] = 1;  
    f(m);  
}
```

'this' argument has type 'const std::map<int, int>', but method is not marked const
mapped_type& operator[](key_type&& __k);

lower_bound(key), upper_bound(key), find(key)

```
std::map<char, int> f { {'c', 20}, {'d', 80}, {'e', 60}};
f['a'] = 10;
f['b'] = 30;

std::cout << f.lower_bound('b')->first << ' '; //>='b'
std::cout << f.upper_bound('b')->first << ' '; //>'b'

std::cout << f.find('c')->second << ' '; //20
std::cout << f.find('r')->second << ' '; //UB
```

Компаратор

```
template <typename Key, typename Value, typename Compare = std::less<Key>>
class map {

};
```

```
int main() {
    std::map<char, int, std::greater<char>> f { {'c', 20}, {'f', 80}, {'e', 60}};
    f['a'] = 10;
    f['b'] = 30;

    for(const auto& x: f) {
        std::cout << x.first << ' ' << x.second << ' ';
    }
}
```

Можно использовать свой компаратор (о классах сравнения и лямбда потом)

```
bool fcomp (char x, char y) {  
    return x > y;  
}  
  
int main() {  
    bool (*fn_pt)(char, char) = fcomp;  
    std::map<char, int, bool (*)(char, char)> f(fn_pt);  
  
    f['a'] = 10;  
    f['b'] = 30;  
  
    for(const auto& x: f) {  
        std::cout << x.first << ' ' << x.second << ' ' << '\n';  
    }  
}
```

Если использовать свой класс в качестве ключа

```
struct myTest{
    int a = 0;
    int b = 3;
    myTest(int a, int b):a(a),b(b) {}
    bool operator < (const myTest& other) const {
        return (a < other.a) || (b < other.b);
    }
};

std::ostream& operator << (std::ostream& out, const myTest& my) {
    return out << my.a << ' ' << my.b << ' ';
}

int main() {
    myTest A(3,5);
    std::map<myTest, char> my;
    my[A] = 'a';
    my[{4,6}] = 'b';
    my[{2,2}] = 'b';
    for(const auto& x: my) {
        std::cout << x.first << ' ' << x.second << '\n';
    }
}
```